

PL00 Laboratory Class :: Planning & Oracle Virtual Box & C Programming Language

Mestrado em Engenharia de
Sistemas Computacionais Críticos

{llf,cag}@isep.ipp.pt

Agenda

1. Local virtual machines using Oracle VirtualBox
2. Compiling and running C applications
3. C Programming Language

Contacts

1. Luis Lino Ferreira
llf@isep.ipp.pt
2. Carlos Gonçalves
cag@isep.ipp.pt

Planning

2026

COMCS ISEP

Su	Mo	Tu	We	Th	Fr	Sa
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

JUL

Su	Mo	Tu	We	Th	Fr	Sa
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

OCT

Su	Mo	Tu	We	Th	Fr	Sa
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30	31					

AUG

Su	Mo	Tu	We	Th	Fr	Sa
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30					

NOV

Su	Mo	Tu	We	Th	Fr	Sa
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30			

SEP

Su	Mo	Tu	We	Th	Fr	Sa
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

DEC

+ COMCS P0 (VirtualBox and C
1 Programming)

2 + COMCS P1 UDP

3 + COMCS P2 TCP

3

4 + COMCS P3 Arduino + UDP + TCP

5 COMCS P4 Arduino Wi-Fi & Bluetooth

6 COMCS P5 BLE GATT

7 COMCS P6 MQTT

8 + COMCS P7 CAN BUS

9 + COMCS P8 NodeRed

10 Apoio Trabalho

11 Apoio Trabalho

12

Programming and environments

In this course, the main programming language to develop network **applications will be C**.

When developing network applications, the way they communicate with each other should not be dependent on the programming language, nor on the underlying operating system. This goal is achieved by defining with no ambiguities the contents of each communication, meaning the **application protocol**.

In addition to timing issues (synchronization), one key factor to ensure the success of communication through an application protocol is specifying accurately **the formats for exchanged data**, and that must be implementation independent.

Oracle VirtualBox

- Another way to access a network environment is to create a local virtual environment with a few nodes. This way, it's possible to test applications in a virtual network without having to use real hardware. to accomplish this the Oracle VirtualBox software can be used. This software allows the creation of virtual machines and virtual networks. Link to install the VirtualBox:
<https://www.virtualbox.org/>
- download the version for your operating system and the "extension pack".

Oracle VirtualBox

- Download the VirtualBox + Extension Pack from: <https://www.virtualbox.org/>



The screenshot shows the Oracle VirtualBox website homepage. At the top left is the VirtualBox logo. To the right are navigation links: Home, Download, Documentation, and Community. The main content area has a light gray background with a subtle geometric pattern. On the left, the text reads: **Powerful open source virtualization** followed by **For personal and enterprise use**. Below this, a paragraph states: "VirtualBox is a general-purpose full virtualization software for x86_64 hardware (with version 7.1 additionally for macOS/Arm and with version 7.2 also for Windows/Arm), targeted at laptop, desktop, server and embedded use." On the right, there is a white box with a blue border. Inside, it says **Get Started** above a blue button labeled **Download**. Below the button, it says "Download VirtualBox binaries and platform packages".

VirtualBox

Home Download Documentation Community

Powerful open source virtualization
For personal and enterprise use

VirtualBox is a general-purpose full virtualization software for x86_64 hardware (with version 7.1 additionally for macOS/Arm and with version 7.2 also for Windows/Arm), targeted at laptop, desktop, server and embedded use.

Get Started

Download

Download VirtualBox binaries and platform packages

Oracle VirtualBox

- Download the VirtualBox + Extension Pack from: <https://www.virtualbox.org/>

VirtualBox Platform Packages

VirtualBox 7.2.2 platform packages

-  [Windows hosts](#)
-  [macOS / Intel hosts](#)
-  [macOS / Apple Silicon hosts](#)
-  [Linux distributions](#)
-  [Solaris hosts](#)
-  [Solaris 11 IPS hosts](#)

Platform packages are released under the terms of the [GPL version 3](#)

VirtualBox Extension Pack

VirtualBox 7.2.2 Extension Pack

This VirtualBox Extension Pack Personal Use and Educational License governs your access to and use of the VirtualBox Extension Pack. It does not apply to the VirtualBox base package and/or its source code, which are licensed under version 3 of the GNU General Public License "GPL").

See our [FAQ](#) for answers to common questions.

VirtualBox Extension Pack Personal Use and Educational License (PUEL)

PUEL License
FAQ

PUEL License
Text

Accept and download

Oracle VirtualBox

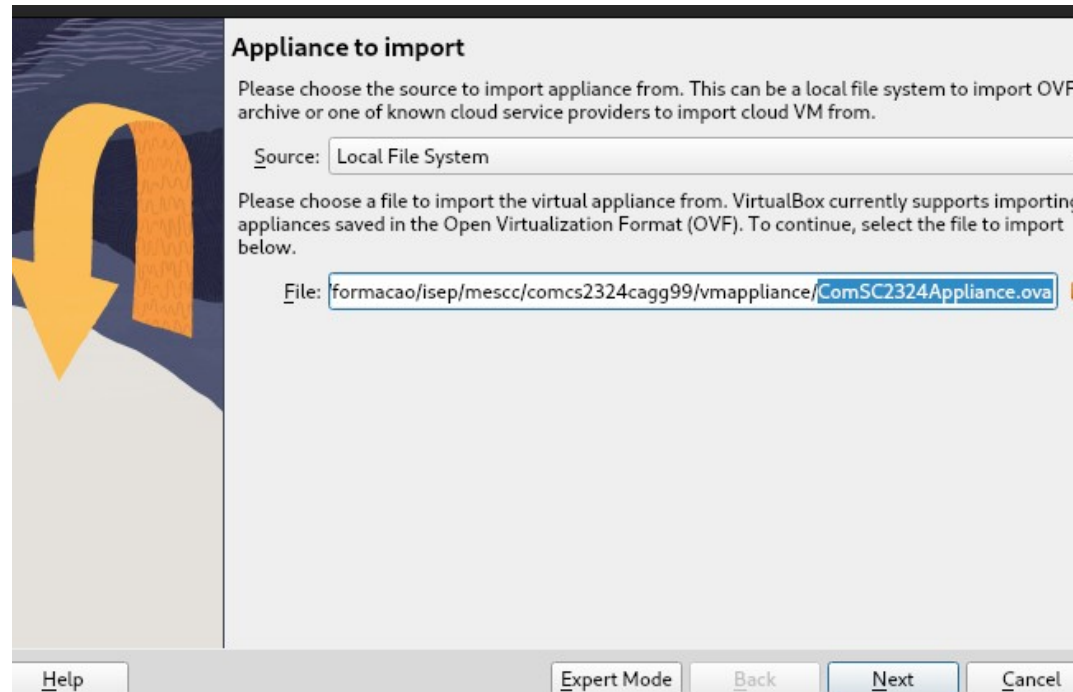
- To speed up the process of creation a virtual environment, there's already a small virtual machine with the necessary software installed. This virtual machine is available in the moodle of the course. Link to the appliance virtual machine:

<https://moodle.isep.ipp.pt/course/view.php?id=3825>

Oracle VirtualBox



File -> Import Appliance

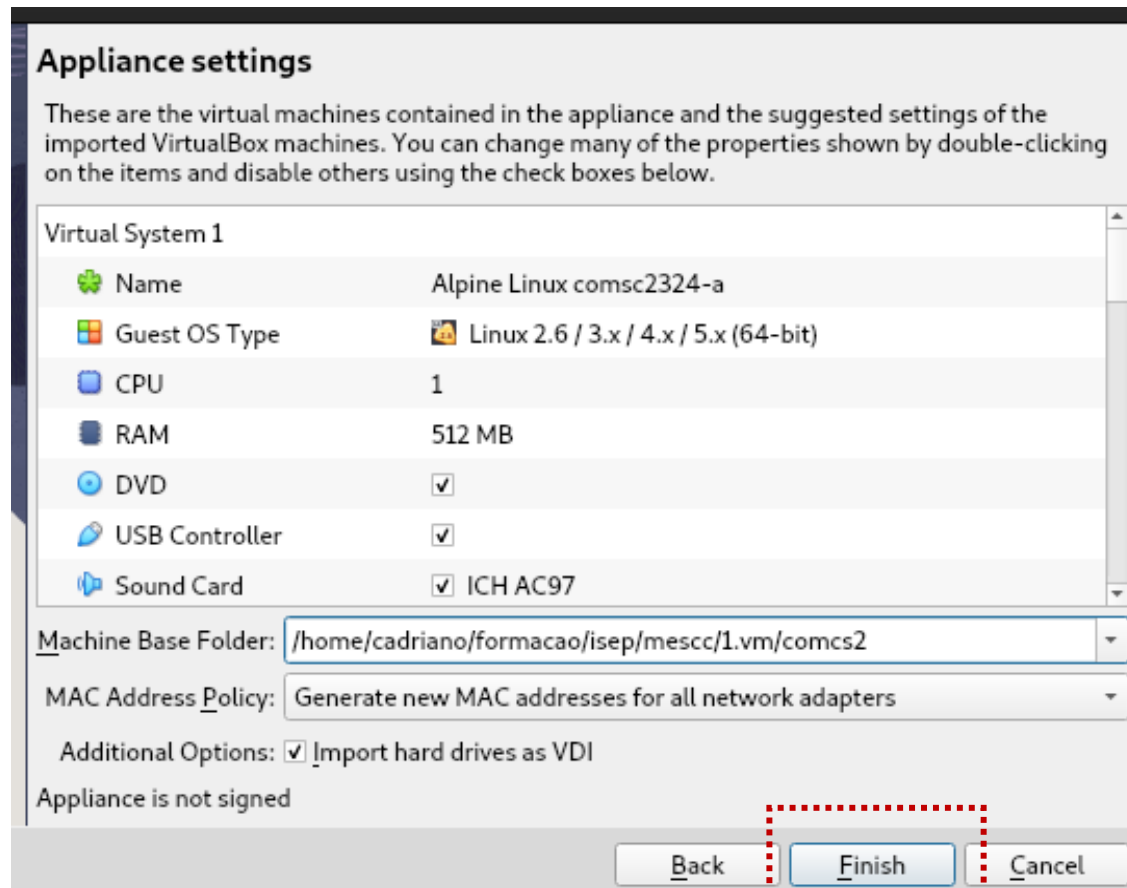


Oracle VirtualBox

Select the base Folder

&

Generate new MAC
Address for all network
Adapters



Oracle VirtualBox

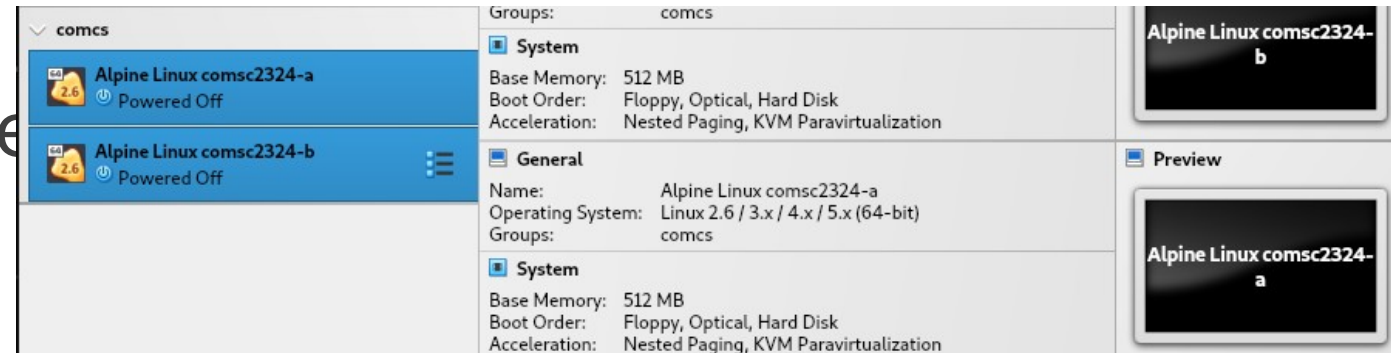
Two machines are available

The screenshot displays the Oracle VM VirtualBox interface. On the left, a list of machines is shown under the 'comcs' group, including 'Alpine Linux comsc2324-a' and 'Alpine Linux comsc2324-b', both in a 'Powered Off' state. The main panel shows the settings for 'Alpine Linux comsc2324-a'. The 'System' tab is selected, showing details like 'Base Memory: 512 MB' and 'Boot Order: Floppy, Optical, Hard Disk'. The 'Network' tab is also visible in the left sidebar. The 'Network' settings panel on the right shows 'Adapter 1' selected, with 'Enable Network Adapter' checked. The 'Attached to' dropdown is set to 'Host-only Adapter', and the 'Name' is 'vboxnet0'. A red dashed box highlights the 'Attached to' and 'Name' fields.

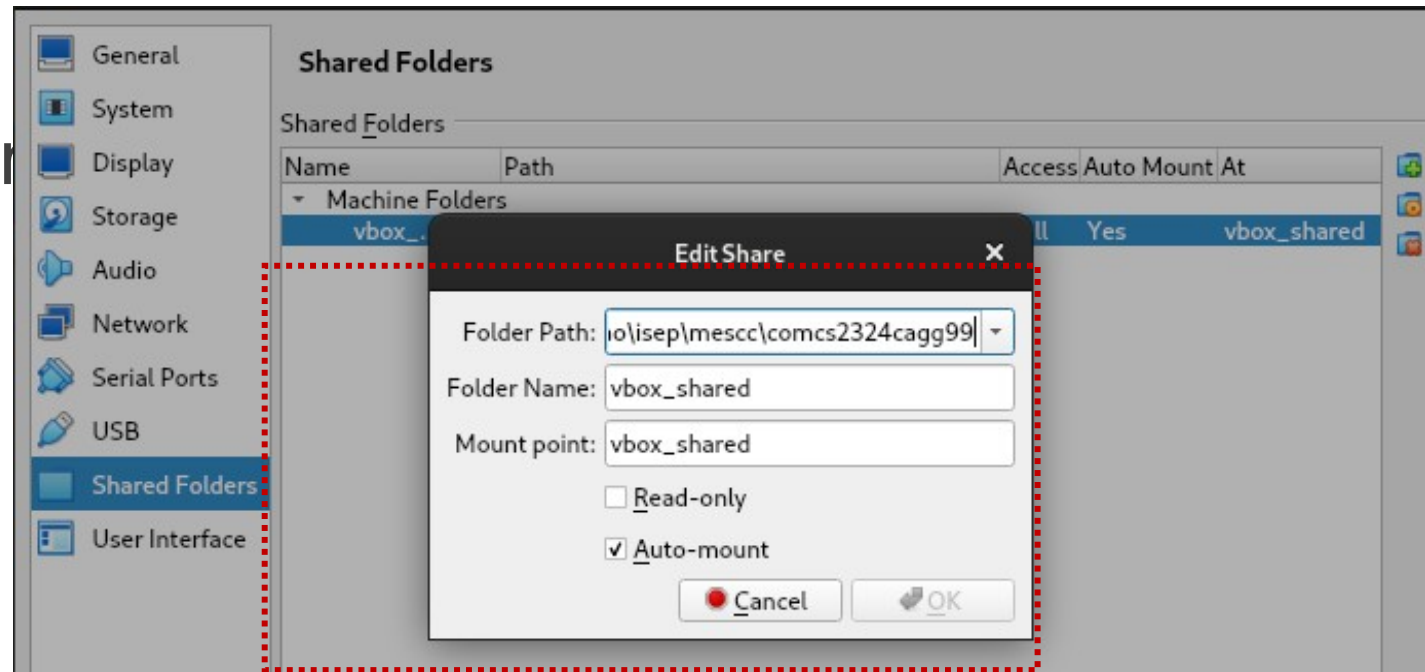
Open the Settings and verify the Network, the "host only Adapter" if It has a name

Oracle VirtualBox

Two machines are available

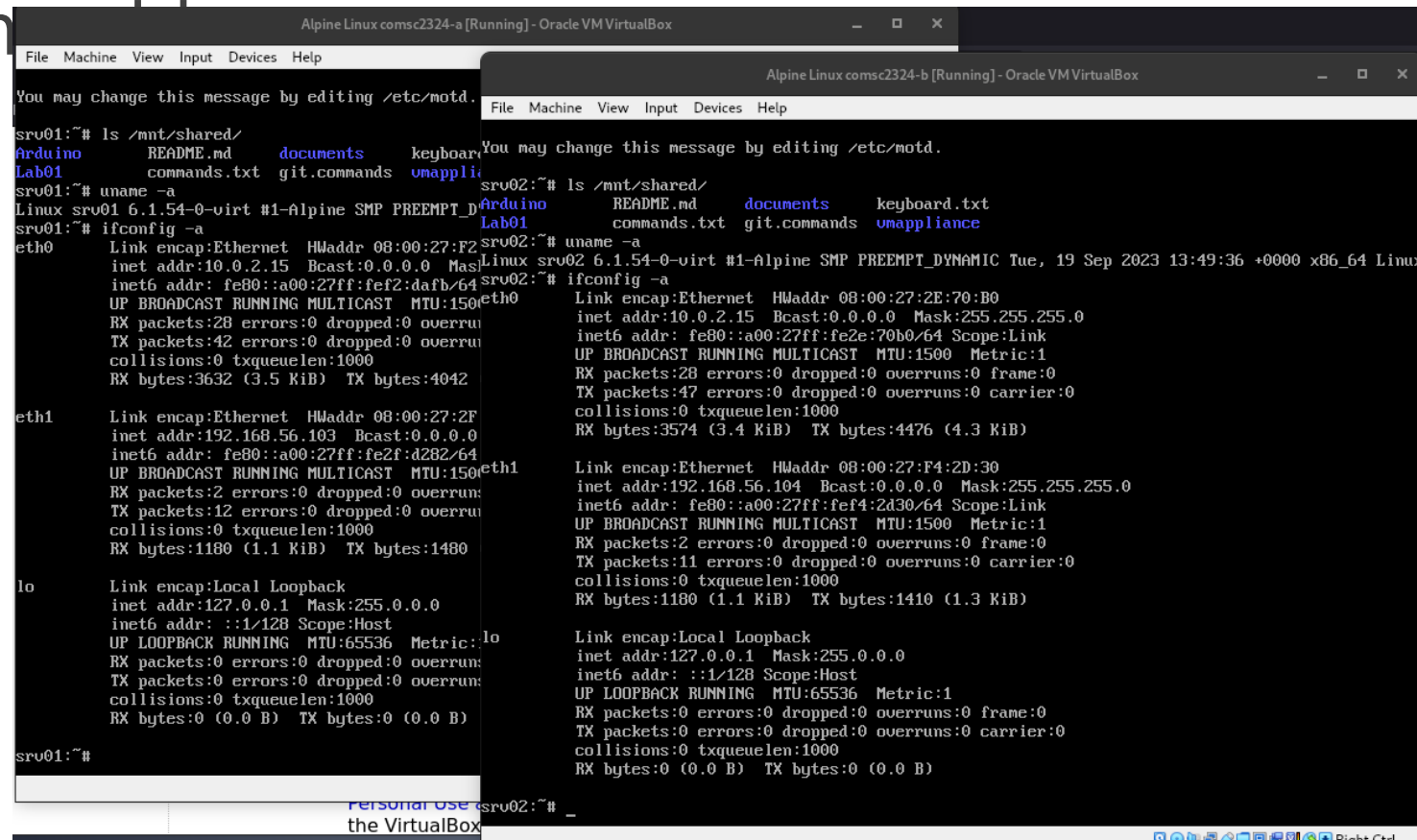


Open the Settings and verify the "Shared Folders", the "Folder Name" and the "Mount Point" must be **vbox_shared**



Oracle VirtualBox

Start the machines: user "root" with "toor" password
“**cd /mnt/shared**” to access the shared folder on the physical machine



The image shows two terminal windows running Alpine Linux inside Oracle VM VirtualBox. The left window is titled 'Alpine Linux comsc2324-a [Running] - Oracle VM VirtualBox' and the right window is titled 'Alpine Linux comsc2324-b [Running] - Oracle VM VirtualBox'. Both windows show the user 'root' at the prompt. The left window shows the command 'ls /mnt/shared/' and its output, which lists files like 'README.md', 'documents', 'keyboard.txt', 'commands.txt', 'git.commands', and 'umappliance'. It also shows the command 'uname -a' and its output, which includes the kernel version 'Linux sru01 6.1.54-0-virt #1-Alpine SMP PREEMPT_DYNAMIC Tue, 19 Sep 2023 13:49:36 +0000 x86_64 Linux'. The right window shows the command 'ls /mnt/shared/' and its output, which lists files like 'README.md', 'documents', 'keyboard.txt', 'commands.txt', 'git.commands', and 'umappliance'. It also shows the command 'uname -a' and its output, which includes the kernel version 'Linux sru02 6.1.54-0-virt #1-Alpine SMP PREEMPT_DYNAMIC Tue, 19 Sep 2023 13:49:36 +0000 x86_64 Linux'. Both windows show the command 'ifconfig -a' and its output, which displays network interface details for 'eth0' and 'eth1'.

```
Alpine Linux comsc2324-a [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
You may change this message by editing /etc/motd.
sru01:~# ls /mnt/shared/
Arduino  README.md  documents  keyboard
Lab01    commands.txt  git.commands  umappliance
sru01:~# uname -a
Linux sru01 6.1.54-0-virt #1-Alpine SMP PREEMPT_DYNAMIC Tue, 19 Sep 2023 13:49:36 +0000 x86_64 Linux
sru01:~# ifconfig -a
eth0: Link encap:Ethernet HWaddr 08:00:27:F2:FE:00
       inet addr:10.0.2.15 Bcast:0.0.0.0 Mask:255.255.255.0
       inet6 addr: fe80::a00:27ff:fe2f:daf6/64 Scope:Link
       UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
       RX packets:28 errors:0 dropped:0 overruns:0 frame:0
       TX packets:42 errors:0 dropped:0 overruns:0 carrier:0
       collisions:0 txqueuelen:1000
       RX bytes:3632 (3.5 KiB) TX bytes:4042

eth1: Link encap:Ethernet HWaddr 08:00:27:2F:FE:00
       inet addr:192.168.56.103 Bcast:0.0.0.0 Mask:255.255.255.0
       inet6 addr: fe80::a00:27ff:fe2f:d282/64 Scope:Link
       UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
       RX packets:2 errors:0 dropped:0 overruns:0 frame:0
       TX packets:12 errors:0 dropped:0 overruns:0 carrier:0
       collisions:0 txqueuelen:1000
       RX bytes:1180 (1.1 KiB) TX bytes:1480

lo: Link encap:Local Loopback
     inet addr:127.0.0.1 Mask:255.0.0.0
     inet6 addr: ::1/128 Scope:Host
     UP LOOPBACK RUNNING MTU:65536 Metric:1
     RX packets:0 errors:0 dropped:0 overruns:0 frame:0
     TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
     collisions:0 txqueuelen:1000
     RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

sru01:~#

Alpine Linux comsc2324-b [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help
You may change this message by editing /etc/motd.
sru02:~# ls /mnt/shared/
Arduino  README.md  documents  keyboard.txt
Lab01    commands.txt  git.commands  umappliance
sru02:~# uname -a
Linux sru02 6.1.54-0-virt #1-Alpine SMP PREEMPT_DYNAMIC Tue, 19 Sep 2023 13:49:36 +0000 x86_64 Linux
sru02:~# ifconfig -a
eth0: Link encap:Ethernet HWaddr 08:00:27:2E:70:B0
       inet addr:10.0.2.15 Bcast:0.0.0.0 Mask:255.255.255.0
       inet6 addr: fe80::a00:27ff:fe2e:70b0/64 Scope:Link
       UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
       RX packets:28 errors:0 dropped:0 overruns:0 frame:0
       TX packets:47 errors:0 dropped:0 overruns:0 carrier:0
       collisions:0 txqueuelen:1000
       RX bytes:3574 (3.4 KiB) TX bytes:4476 (4.3 KiB)

eth1: Link encap:Ethernet HWaddr 08:00:27:F4:2D:30
       inet addr:192.168.56.104 Bcast:0.0.0.0 Mask:255.255.255.0
       inet6 addr: fe80::a00:27ff:fef4:2d30/64 Scope:Link
       UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
       RX packets:2 errors:0 dropped:0 overruns:0 frame:0
       TX packets:11 errors:0 dropped:0 overruns:0 carrier:0
       collisions:0 txqueuelen:1000
       RX bytes:1180 (1.1 KiB) TX bytes:1410 (1.3 KiB)

lo: Link encap:Local Loopback
     inet addr:127.0.0.1 Mask:255.0.0.0
     inet6 addr: ::1/128 Scope:Host
     UP LOOPBACK RUNNING MTU:65536 Metric:1
     RX packets:0 errors:0 dropped:0 overruns:0 frame:0
     TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
     collisions:0 txqueuelen:1000
     RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

sru02:~#
```

Compiling and Running C Programs

- - Source code can be created by using a text editor like **vim** or **nano** so, by running a command like:

➤ **vim SOURCE-FILE.c** or **nano SOURCE-FILE.c**

- - The source file (SOURCE-FILE.c) can then be compiled using gcc (GNU Compiler Collection):

➤ **gcc -Wall -Wextra SOURCE-FILE.c -o EXECUTABLE-FILE**

- If no -o option is used, then an executable file named **a.out** will be created. In case there are other functions used that are defined in other *.c files, then these need to be added to the compilation process.

➤ **gcc -Wall -Wextra OTHER-SOURCE-FILES.c SOURCE-FILE.c -o EXECUTABLE-FILE**

- - To run the application, just call it from the command line: **./EXECUTABLE-FILE**

Compiling and Running C Programs

a single Makefile
to facilitate the process:

```
# variables
INCLUDES =
SOURCES = main.c
OBJFILES = main.o
EXEC = prog.bin
#Compilation parameters
CP = -Wall -Wextra
# regras de sufixo
.SUFFIXES : .c .o

# transform .c into .o ; $< -- filename
.c.o:
    gcc -g ${CP} -c $<

# main program
${EXEC}: ${OBJFILES}
    gcc -g ${CP} -o ${EXEC} ${OBJFILES}

# run the program
run: ${EXEC}
    ./${EXEC}

#keep only the source files
clean:
    rm ${OBJFILES} ${EXEC}
```

C Programing Language

Standard input and output functions (i.e. printf + others)

main function

flag success to
operating system

```
#include <stdio.h>          /* comment */  
  
int main(void)  
{  
    printf("Hello\n");  
    printf("Welcome to the Course!\n");  
  
    return 0;  
}
```

“end”

```
Hello  
Welcome to the Course!
```

C Programing Language

```
#include <stdio.h>
```

```
int main(void)
```

```
{
```

```
    int a, b;
```

//create two integer variables, "a" and "b"

```
    printf("Enter two numbers: ");
```

```
    scanf("%i %i", &a, &b);
```

//read two integer numbers into "a" and "b"

```
    printf("%i - %i = %i\n", a, b, a - b);
```

```
    return 0;
```

```
}
```

write "a", "b" and "a-b" in
the format specified

```
Enter two numbers: 21 17
21 - 17 = 4
```

C Programing Language

- C supports different kinds of integers
- maxima and minima defined in “limits.h”

type	format	bytes	minimum	maximum
char	%c	1	CHAR_MIN	CHAR_MAX
signed char	%c	1	SCHAR_MIN	SCHAR_MAX
unsigned char	%c	1	0	UCHAR_MAX
short [int]	%hi	2	SHRT_MIN	SHRT_MAX
unsigned short	%hu	2	0	USHRT_MAX
int	%i	2 or 4	INT_MIN	INT_MAX
unsigned int	%u	2 or 4	0	UINT_MAX
long [int]	%li	4	LONG_MIN	LONG_MAX
unsigned long	%lu	4	0	ULONG_MAX

C Programing Language

- C supports different kinds of reals
- maxima and minima are defined in “float.h”

type	format			bytes	minimum	maximum
float	%f	%e	%g	4	FLT_MIN	FLT_MAX
double	%lf	%le	%lg	8	DBL_MIN	DBL_MAX
long double	%Lf	%Le	%Lg	10	LDBL_MIN	LDBL_MAX

C Programing Language

- The cast operator *temporarily* changes the type of a variable

if either operand is a double,
the other is automatically
promoted

integer division is done here,
the result, 1, is changed to a
double, 1.00000

```
int main(void)
{
    int    i = 5, j = 4;
    double f;

    f = (double)i / j;
    f = i / (double)j;
    f = (double)i / (double)j;
    f = (double)(i / j);

    return 0;
}
```

C Programing Language

▪ Calling a function (by value)

```
#include <stdio.h>
void func_a(int );
int main() {
    int x = 3;
    printf("Value before call func_a: %d\n", x);
    func_a(x);
    printf("Value after pass-by-value func_a: %d\n", x);
    printf("\n");
    return 0;
}
//pass by value
void func_a(int a) {
    a = 5;
    printf("func_a=%d\n", a);
}
```

Output:
Value before call func_a: 3
func_a=5
Value after pass-by-value func_a: 3

C Programing Language

▪ Pointers & Calling a function (by reference but ...)

```
#include <stdio.h>
void func_b(int* );
int main() {
    int x = 3;
    int* xptr = &x;
    printf("Value before call  func_b: %d\n", x);
    printf("Pointer before function call  func_b: %p\n", (void*)xptr);
    func_b(xptr);
    printf("Value after call  func_b: %d\n", x);
    printf("Pointer after function call  func_b: %p\n", (void*)xptr);
    return 0;
}

//but ... Dereferencing...
//the pointer is passed as a pass-by-value argument.
void func_b(int* a) {
    int b = 9;
    *a = 5;
    a = &b;  //only applied inside the function
    printf("func_b=%d\n", *a);
}
```

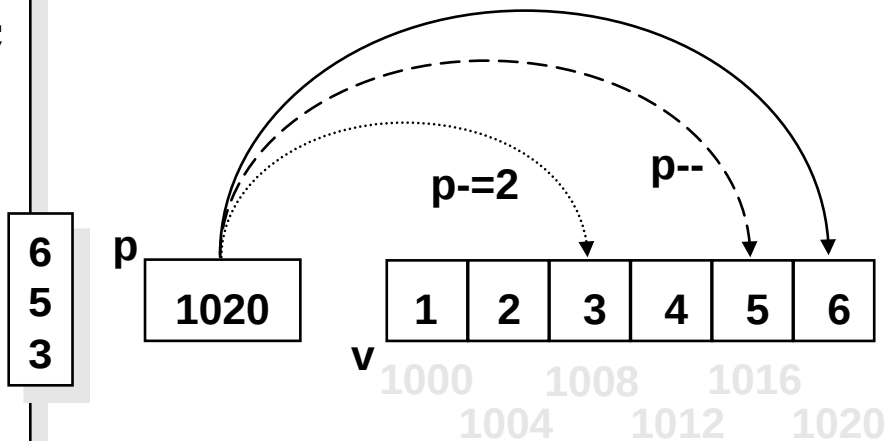
Output:

```
Value before call  func_b: 3
Pointer before function call
func_b: 0x7ffdcfd82afc
func_b=9
Value after call  func_b: 5
Pointer after function call
func_b: 0x7ffdcfd82afc
```

C Programing Language

Pointers and arrays: Scaling not only happens when addition is done, it happens with subtraction too

```
long v[6] = { 1,2,
              3,4,5,6 };
long *p;
p = v + 5;
printf("%ld\n", *p);
p--;
printf("%ld\n", *p);
p -= 2;
printf("%ld\n", *p);
```



C Programing Language

Structure instances may be initialised using braces (as with arrays)

```
int  primes[7] = { 1, 2, 3, 5, 7, 11, 13 };

struct  Date    bug_day = { 1, 1, 2000 };

struct  Book    k_and_r = {
    "The C Programming Language 2nd edition",
    "Brian W. Kernighan and Dennis M. Ritchie",
    31.95,
    "0-13-110362-8"
};
```

```
struct Date
{
    int  day;
    int  month;
    int  year;
};
```

```
struct Book
{
    char    title[80];
    char    author[80];
    float   price;
    char    isbn[20];
};
```

**INSTITUTO SUPERIOR DE ENGENHARIA
DO PORTO**

**Rua Dr. António Bernardino de Almeida,
431**

4249-015 Porto, Portugal

—
T(+351) 228 340 500

F (+351) 228 321 159

—
www.isep.ipp.pt



WWW:
<http://www.isep.ipp.pt/mescc>
e-mail: mescc@isep.ipp.pt